EEUC201 · ANALOG AND DIGITAL ELECTRONICS

Flip-Flops, Timers and Counters

S-R · D · T · J-K Flip-Flops — Clocked Design — Registers — Asynchronous & Synchronous Counters

Sequential logic building blocks: memory elements, storage registers, and binary counting circuits

OVERVIEW

Agenda

Basic Flip-Flops

S-R, D, T, and J-K flip-flop operation & truth tables

01

Clocked Flip-Flop Design

Excitation tables and the sequential design procedure

02

Registers

Shift register configurations for data storage & transfer

03

Asynchronous Counters

Ripple counters built from cascaded flip-flops

04

Synchronous Counters

Common-clock counters with parallel state transitions

05

SEQUENTIAL LOGIC BASICS

What Is a Flip-Flop?

- A bistable memory element that stores one bit of binary data (0 or 1).
- Unlike a latch, a flip-flop changes state only in response to a clock edge (edge-triggered), not a level.
- Built from cross-coupled logic gates (NAND/NOR) forming a feedback loop with two stable outputs, Q and Q'.
- Four standard types: S-R, D, T, and J-K — each differs in how inputs control the next state.

LATCH vs FLIP-FLOP

Latch

Level-sensitive — output follows input while the enable is active.

Flip-Flop

Edge-triggered — output changes only at the rising or falling edge of the clock pulse.

Why it matters:

Edge-triggering prevents race conditions in multi-stage circuits, allowing predictable, synchronized data transfer between registers.

FLIP-FLOP TYPES

S-R Flip-Flop

Set-Reset Latch

- The simplest flip-flop: two inputs, S (Set) and R (Reset).
- S = 1, R = 0 → Q sets to 1
- S = 0, R = 1 → Q resets to 0
- S = 0, R = 0 → Q holds its previous state
- S = 1, R = 1 → Invalid / forbidden state (both outputs forced to same level)

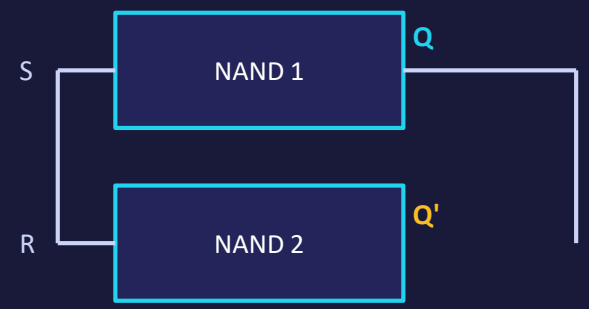
Characteristic equation:

$$Q(next) = S + R' \cdot Q \quad (\text{with } S \cdot R = 0 \text{ constraint})$$

CHARACTERISTIC TABLE

S	R	Q(next)	Comment
0	0	Q	Hold
0	1	0	Reset
1	0	1	Set

NAND-GATE IMPLEMENTATION



D Flip-Flop

Data / Delay Flip-Flop

- Has a single data input, D, plus a clock input.
- At the active clock edge, the output Q simply takes on the value present at D.
- Eliminates the invalid state problem of the S-R flip-flop — D is never allowed to be both 1 and 0.
- Effectively delays the input by one clock period, hence the name 'Delay' flip-flop.
- Widely used in registers, memory cells, and pipeline stages.

CHARACTERISTIC TABLE

D	Q(next)
0	0
1	1

Characteristic equation: Q(next) = D

FLIP-FLOP TYPES

T Flip-Flop

Toggle Flip-Flop

- Has a single input, T (Toggle), plus a clock input.
- T = 0 → output holds its previous state.
- T = 1 → output toggles (complements) on every active clock edge.
- Often derived from a J-K flip-flop by tying J and K together (J = K = T).
- Ideal building block for binary counters, since successive bits toggle at half the frequency of the previous stage.

CHARACTERISTIC TABLE

T	Q(next)	Comment
0	Q	Hold
1	Q'	Toggle

Characteristic equation: $Q(next) = T \oplus Q$

FREQUENCY DIVISION (T = 1)

FLIP-FLOP TYPES

J-K Flip-Flop

The Universal Flip-Flop

- Improves on the S-R flip-flop by resolving the invalid input combination.
- J = 0, K = 0 → Hold (no change)
- J = 0, K = 1 → Reset (Q = 0)
- J = 1, K = 0 → Set (Q = 1)
- J = 1, K = 1 → Toggle (Q complements) — the previously forbidden case now has a defined, useful result.
- Called 'universal' because D, T, and S-R behaviour can all be derived from it.

CHARACTERISTIC TABLE

J	K	Q(next)	Comment
0	0	Q	Hold
0	1	0	Reset
1	0	1	Set

Q(next) = J·Q' + K'·Q

No invalid state exists — every one of the four input combinations produces a well-defined, useful next state.

FLIP-FLOP COMPARISON

Four Types at a Glance

Type	Inputs	Characteristic Eqn.	Invalid State?	Typical Use
S-R	S, R	Q(next) = S + R'Q	Yes (S=R=1)	Basic latch, debounce
D	D	Q(next) = D	None	Registers, memory cells
T	T	Q(next) = T⊕Q	None	Binary counters

D and T flip-flops can each be built from a J-K flip-flop: tie J = D, K = D' for a D-type; tie J = K = T for a T-type.

SEQUENTIAL DESIGN

Clocked Flip-Flop Design Procedure

1 **State Diagram**
Define required states & transitions

→

2 **State Table**
Tabulate present state, input, next state

→

3 **Excitation Table**
Determine flip-flop inputs for each transition

→

4 **K-Maps**
Simplify each flip-flop input equation

→

5 **Logic Diagram**
Implement equations with gates & flip-flops

EXCITATION TABLES (INPUTS NEEDED FOR A GIVEN TRANSITION)

S-R

Q	Q(next)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

J-K

Q	Q(next)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

D

Q	Q(next)	D
0	0	0
0	1	1
1	0	0
1	1	1

T

Q	Q(next)	T
0	0	0
0	1	1
1	0	1
1	1	0

'X' denotes a don't-care condition — either input value produces the required transition, simplifying the resulting K-map.

REGISTERS

Shift Registers

- A register is a group of flip-flops (typically D-type) used to store multiple bits of data.
- A shift register additionally moves stored bits left or right on each clock pulse — used for serial-to-parallel conversion, delay lines, and arithmetic operations.

SISO

Serial-In, Serial-Out

Data enters and exits one bit at a time — used purely as a time delay line.

SIPO

Serial-In, Parallel-Out

Data loaded serially, then all bits read out simultaneously — common in UART receivers.

PISO

Parallel-In, Serial-Out

All bits loaded at once, then shifted out one at a time — used in UART transmitters.

PIPO

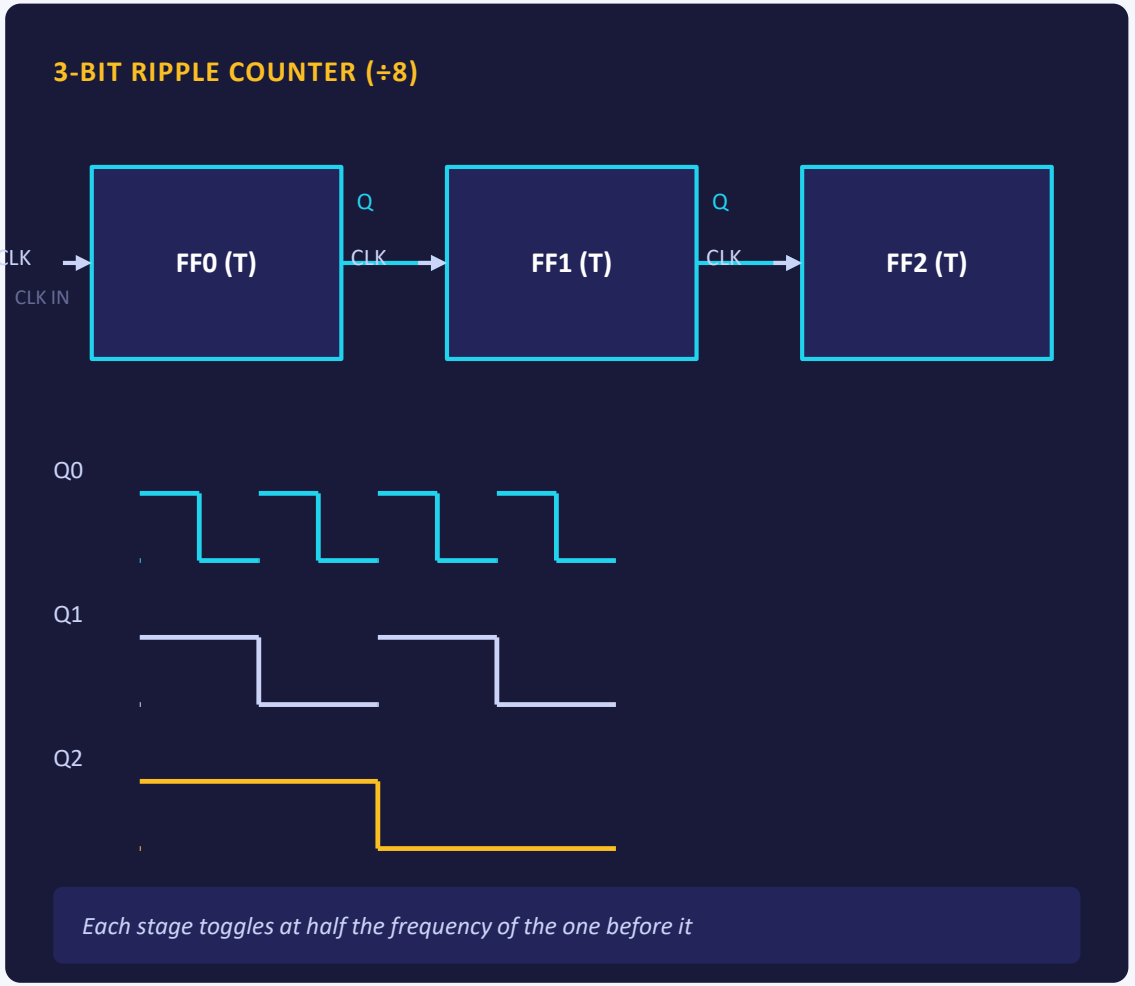
Parallel-In, Parallel-Out

Bits loaded and read out simultaneously — acts as a simple buffer register.

Asynchronous (Ripple) Counters

Ripple-Clocked Design

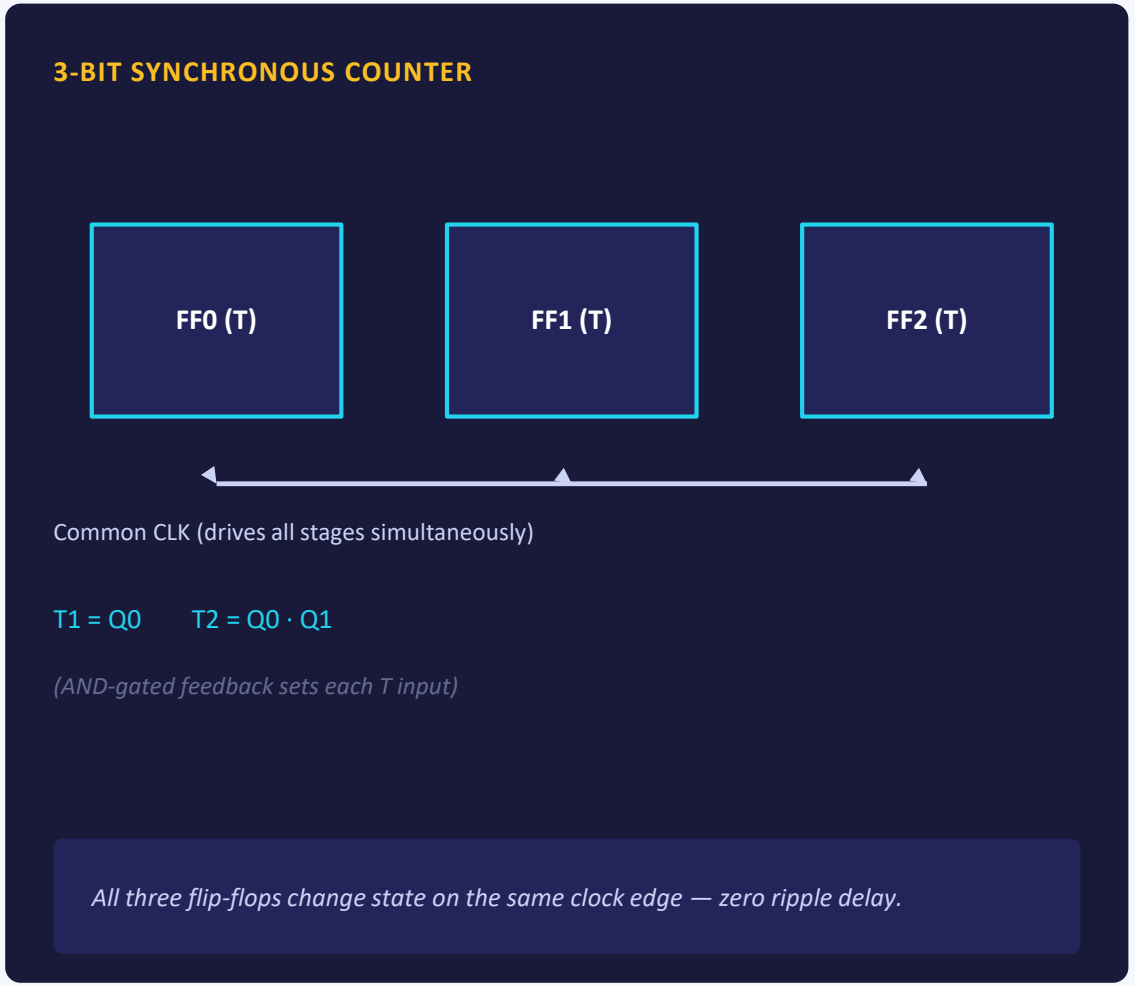
- Only the first flip-flop is driven by the external clock; each subsequent flip-flop is clocked by the Q output of the previous stage.
- Built from T (or J-K wired as T) flip-flops toggling on every input pulse.
- The change 'ripples' through the chain — each flip-flop toggles slightly after the one before it, so delays accumulate stage by stage.
- n flip-flops divide the input frequency by 2^n and count from 0 to 2^n-1.
- Simple to build, but slower — not suitable for high-speed applications.



Synchronous Counters

Common-Clock Design

- Every flip-flop shares the same clock signal, so all outputs change at the same instant.
- Combinational logic (AND gates or J-K input equations) determines each flip-flop's next-state inputs from the current count.
- No cumulative ripple delay — total propagation delay equals that of a single flip-flop, regardless of counter length.
- Requires more design effort and gates than asynchronous counters, but supports much higher clock speeds.
- Preferred in high-speed digital systems and FPGA/ASIC designs.



COUNTER COMPARISON

Asynchronous vs Synchronous Counters

Aspect	Asynchronous (Ripple)	Synchronous
Clock connection	Only first FF gets external clock	All FFs share the same clock
Speed	Slower — delays accumulate stage by stage	Faster — single flip-flop delay only
Design complexity	Simple, minimal extra logic	More complex, needs combinational logic
Output glitches	Possible during transitions (ripple)	Clean, simultaneous transitions
Typical use	Low-speed, simple counting tasks	High-speed digital systems

SUMMARY

Key Takeaways

Flip-Flop Types

S-R (simple, has invalid state), D (delay), T (toggle), and J-K (universal, no invalid state)

Clocked Design

State diagram → state table → excitation table → K-maps → logic diagram

Registers

SISO, SIPO, PISO, and PIPO shift registers store and move multi-bit data

Counters

Asynchronous counters ripple and accumulate delay; synchronous counters share one clock for speed